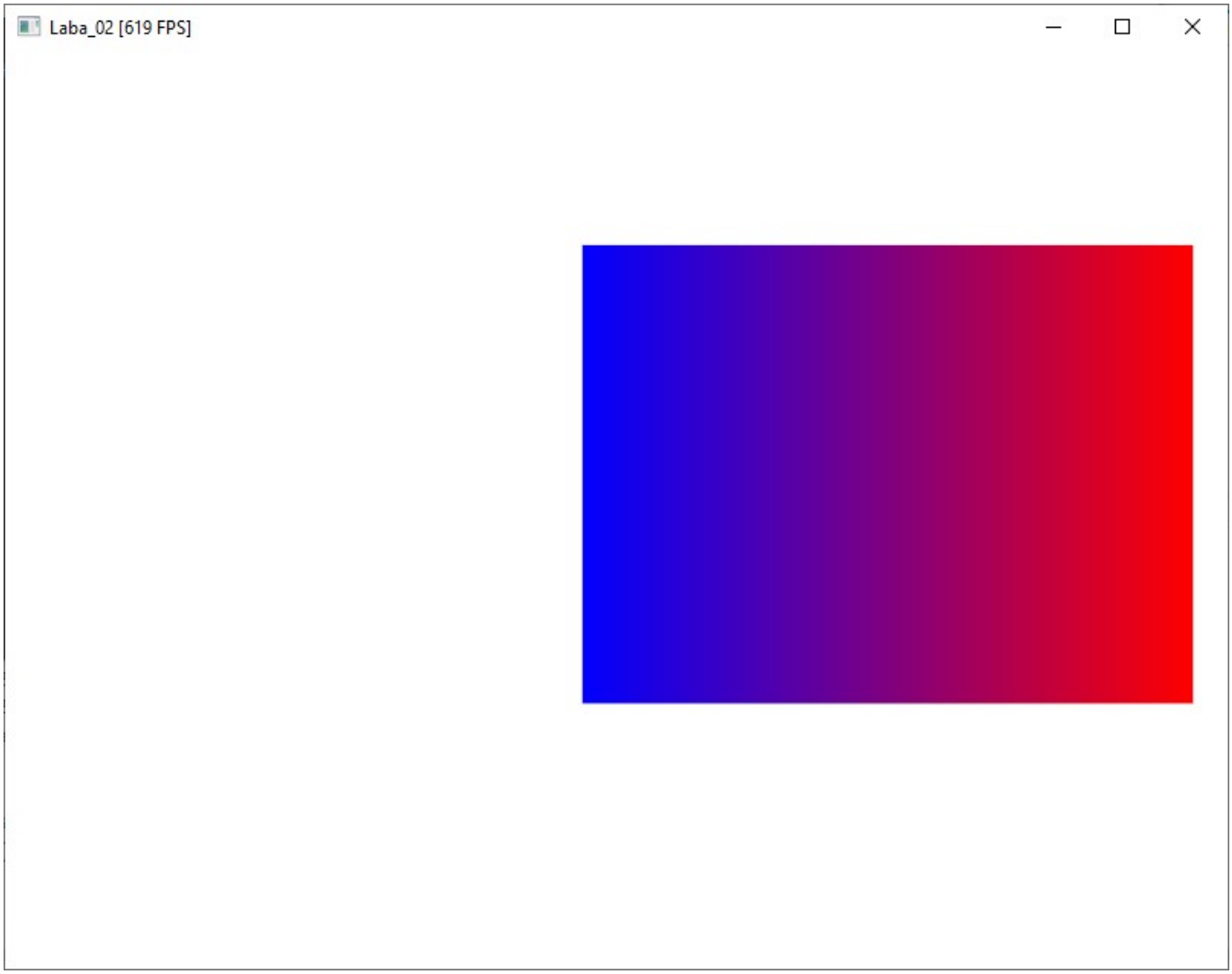
Лабораторная работа №02

Передача данных в шейдерную программу.

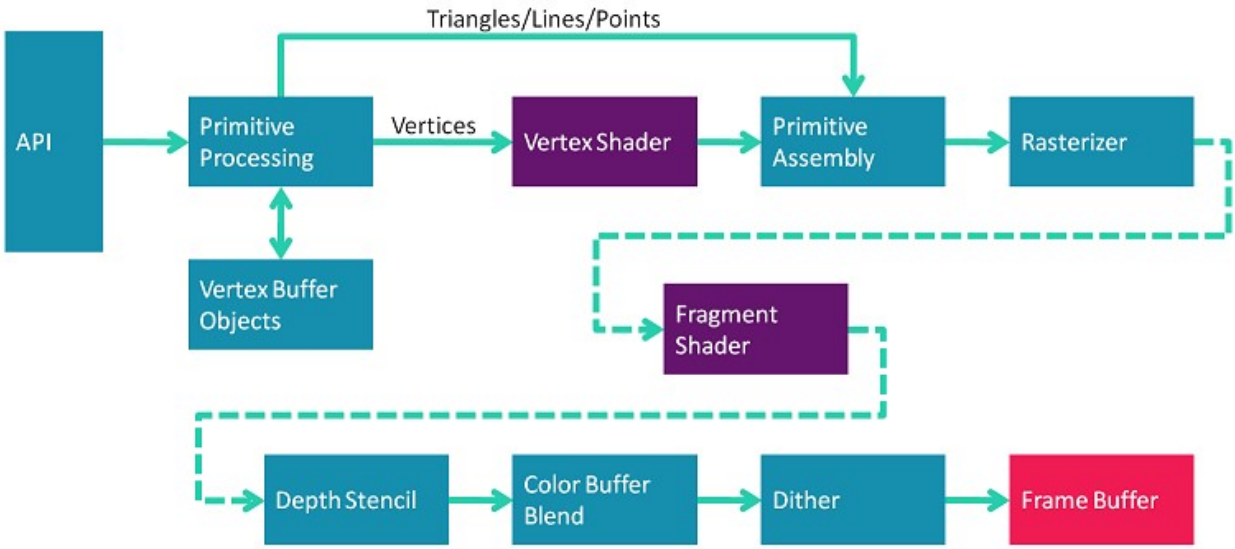
Целью лабораторной работы №02 является изучение способов передачи данных между основной программой и шейдерами. В частности, рассматривается назначение uniform-переменных и способов их установки из основной программы, а также механизм передачи данных между вершинным и фрагментным шейдером. Используя полученные сведения необходимо вывести на экран перемещающийся прямоугольник, раскрашенный в соответствии с заданием к лабораторной работе.



При реализации программы необходимо также реализовать функцию для вычисления количества FPS (кадров в секунду) и функцию для определения времени симуляции. Симуляция передвижения прямоугольника должна происходить с предсказуемой, заранее заданной скоростью, не зависящей от производительности программы.

Использование uniform-переменных.

В современной версии OpenGL результат вывода графических объектов на экран определяется шейдерами, в частности: вершинный шейдер занимается обработкой каждой вершины, определяя, таким образом, её положение на экране, в то время как фрагментный шейдер занимается обработкой каждого фрагмента (пикселя), определяя его цвет.



В OpenGL шейдерные программы пишутся на специальном языке программирования GLSL и, как и любые другие программы, представляют собой алгоритм, в соответствии с которым обрабатываются вершины и фрагменты. Для работы любого алгоритма, как правило, необходимы входные данные. Данные для шейдера передаются из основной программы (программы, использующей OpenGL и указанные шейдеры для построения изображения на экране).

Одним из вариантов передачи данных из основной программы в шейдер является использование uniform-переменных. Uniform-переменные, это глобальные переменные объявленные в вершинном или фрагментном шейдере с дополнительным квалификатором uniform. Каждый шейдер может иметь произвольное количество uniform-переменных. Процесс установки uniform-переменных, таким образом, реализуется в соответствии со следующей схемой:

1. **Выбор шейдера в качестве текущего.** Значение переменной можно установить только для текущего шейдера, поэтому прежде всего необходимо выбрать шейдер в качестве текущего. Следует отметить, что с целью оптимизации можно не выбирать шейдер, если он уже является активным (для этого рекомендуется отслеживать эту ситуацию, например, с помощью статического поля класса).

2. **Получение индекса (location) uniform-переменной по её имени.** Для того чтобы иметь возможность установить значение переменной необходимо знать её индекс в шейдере. Индекс переменной можно получить по её имени используя функцию glGetUniformLocation. Индекс переменной определяется на этапе линковки и не меняется в процессе существования шейдерной программы. В связи с этим крайне желательно не вызывать данную функцию постоянно, а запомнить индекс переменной после первого вызова указанной функции, в ассоциативном контейнере (std::map). Информация о функции glGetUniformLocation доступна по следующей ссылке:

   https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glGetUniformLocation.xhtml

3. **Установка значения uniform-переменной.** Непосредственно для установки значения uniform-переменной используется одна из функций, выбор которой зависит от типа uniform-переменной. Например, для установки вектора из четырех вещественных чисел используется функция glUniform4fv, для установки вектора из двух вещественных величин – функция glUniform2fv, а для установки матрицы 4x4 используется функция glUniformMatrix4fv. Подробнее о функциях установки значения uniform-переменных можно узнать по следующим ссылкам:

   https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/glUniform.xhtml

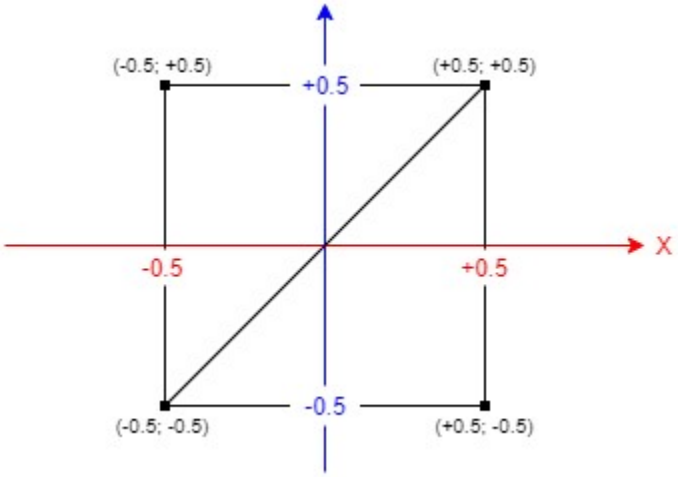
Для работы с uniform-переменными в лабораторной работе необходимо изменить ранее реализованный класс Shader. Следует обратить внимание, что для установки uniform-переменных используются перегруженные функции, а в качестве параметров выступают классы библиотеки glm, что позволяет более удобным способом хранить, передавать и модифицировать необходимые данные. Сам класс, на данный момент, должен выглядеть следующим образом:

```
// КЛАСС ДЛЯ РАБОТЫ С ШЕЙДЕРОМ
class Shader
{
public:
	// загрузка шейдера из внешних файлов
	bool load(std::string veftexShaderFilename, std::string fragmentShaderFilename);

	// выбор шейдера в качестве текущего
	void activate();

	// отключение шейдера
	static void  deactivate();

	// установка uniform-переменных
	void setUniform(std::string name, int value);
	void setUniform(std::string name, float value);
	void setUniform(std::string name, glm::vec2 &value);
	void setUniform(std::string name, glm::vec4 &value);
	void setUniform(std::string name, glm::mat4 &value);

private:
	// создание шейдерного объекта указанного типа
	// и загрузка исходного текста шейдера из указанного файла
	GLuint createShaderObject(GLenum shaderType, std::string filename);

	// получение индекса (location) uniform-переменной
	GLuint getUniformLocation(std::string name);

private:
	// id "текущей" выбранной шейдерной программы
	// используется чтобы повторно не активировать шейдерную программу,
	// если она уже активирована
	static GLuint currentProgram;

	// шейдерная программа (шейдер)
	GLuint program;

	// "справочник" uniform-переменных,
	// который определяет ранее запомненное соответствие
	// между именем uniform-переменной и её индексом (location)
	std::map <std::string, GLuint> uniforms;

};
```

Работа вершинного шейдера.

Для построения изображения на экране используется меш – это математическое описание геометрической формы, состоящее из множества полигонов (как правило треугольников). Каждый треугольник, в свою очередь, описывается всеми своими вершинами, каждая из которых состоит из множества атрибутов.

В данной лабораторной работе для вывода меша на экран используется функция drawObject. Данная функция генерирует и выводит на экран квадрат, состоящий из двух треугольников. Каждая вершина сгенерированного меша имеет только один атрибут – вектор из двух компонент, который определяет положение каждой вершины в соответствии с рисунком, приведенным ниже:



Для вывода графического объекта на экран, как правило, используется следующая последовательность действий:

1. **Выбор шейдера в качестве текущего.** Прежде всего, необходимо выбрать шейдер (совокупность вершинного и фрагментного шейдеров), который будет отвечать за вывод объекта на экран. Выбранный шейдер задает алгоритм преобразования вершин и алгоритм раскраски фрагментов, то есть, как правило, отвечает за реализацию какого-либо графического эффекта. Один и тот же шейдер может использоваться для вывода нескольких объектов.

2. **Установка uniform-переменных.** Далее необходимо установить значения используемых в шейдере uniform-переменных. Данные значения будут использоваться в процессе вывода всего объекта, то есть данные значения будут общими для всех вершин и всех фрагментов одного объекта.

3. **Вывод самого меша на экран.** Вывод меша подразумевает передачу в OpenGL всех вершин объекта с указанием всех атрибутов каждой вершины. Процесс вывода меша будет подробно рассмотрен в одной из следующий лабораторных работ. В данный лабораторной работе для этих целей используется функция drawObject из предыдущей лабораторной работы.

После того, как основная программа выполнила все действия, указанные выше, управление передается видеокарте. В частности, видеокарта вызывает вершинный шейдер для каждой вершины (для приведенного выше примера вершин будет шесть, две вершины являются идентичными). При этом в вершинный шейдер при каждом вызове передаются следующие данные:

1. **Атрибуты вершин.** Атрибуты вершин – это глобальные переменные вершинного шейдера, объявленные с дополнительным квалификатором ***in***. В данном случае у каждой вершины всего один атрибут – это двухмерный вектор, содержащий позицию вершины в соответствии с рисунком.

2. **Uniform-переменные.** Uniform-переменные – это глобальные переменные вершинного шейдера, объявленные с дополнительным квалификатором ***uniform***. В вершинный шейдер передается ранее установленное значение для каждой uniform-переменной.

3. **Дополнительные встроенные переменные.** Данные переменные генерируются самой видеокартой и могут использоваться в процессе преобразования вершин.

Вершинный шейдер, получив все данные, должен выполнить две основные задачи:

1. **Определить данные для фрагментного шейдера.**

   Вершинный шейдер может вычислить и передать во фрагментный шейдер некоторые данные, если эти данные могут быть полезны в процессе работы фрагментного шейдера, например, для вычисления цвета фрагмента.

   Для этого в вершинном шейдере необходимо объявить глобальную переменную с квалификатором out - данная переменная будет является выходной для вершинного шейдера. Вершинный шейдер обязан присвоить значение для всех своих выходных переменных.

2. **Определить позицию вершины на экране, заполнив встроенную переменную gl_Position.**

   Этот этап, в отличие от предыдущего, является обязательным.

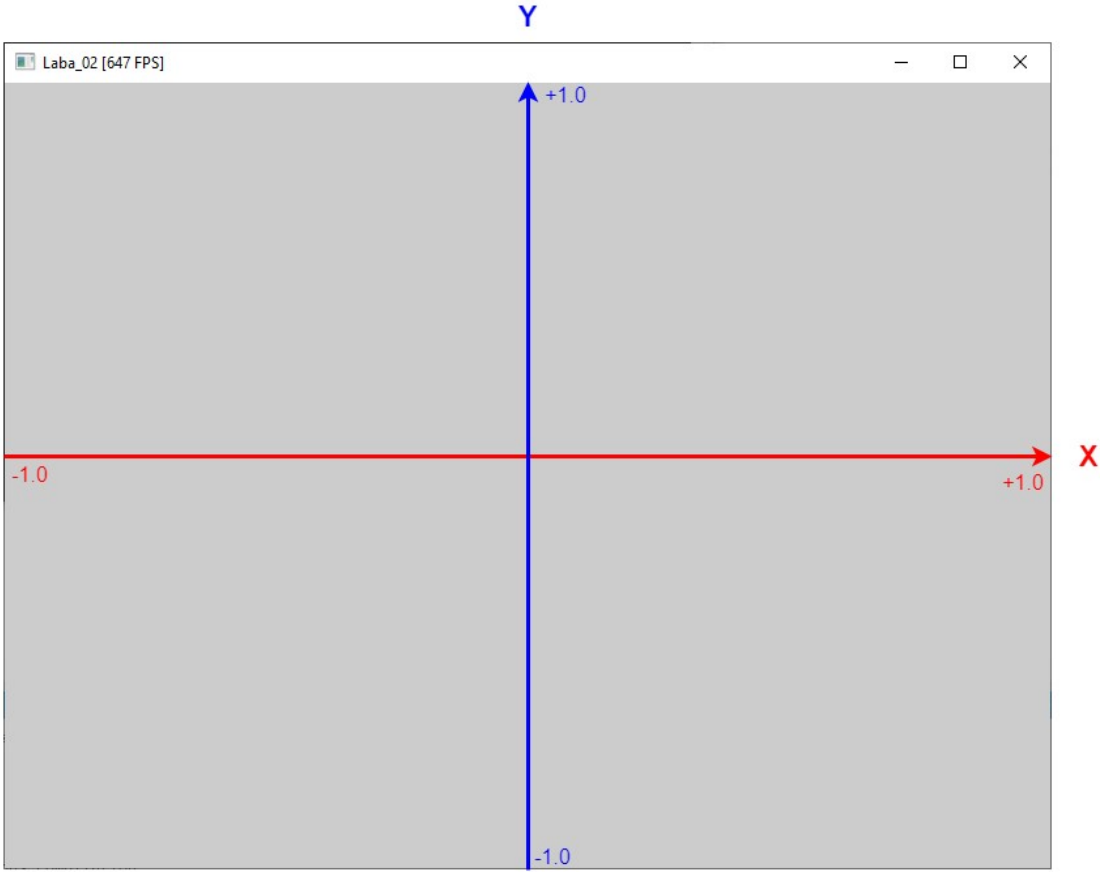
   Переменная gl_Position это вектор из четырех компонент (vec4), она является встроенной (её не надо объявлять самостоятельно), и она будет использоваться библиотекой OpenGL для принятия решения о том, в каком месте экрана расположить данную вершину.

   Работа с данной переменной более подробно будет рассмотрена в следующей лабораторной работе, пока же необходимо знать, что если последняя компонента w равна единице (как в приведенном ниже примере), то остальные компоненты можно трактовать следующим образом:

   X – определяет положение по горизонтали. Данная переменная должна меняться в диапазоне от -1, до +1. При этом если координата равна -1, то вершина лежит на левой границе области вывода (максимально слева), а если переменная равна +1, то вершина лежит на правой границе области вывода (максимально справа);

   Y – определяет положение по вертикали. Данная переменная должна меняться в диапазоне от -1, до +1. При этом если координата равна -1, то вершина лежит на нижней границе области вывода (максимально низко), а если переменная равна +1, то вершина лежит на верхней границе области вывода (максимально высоко);

   Z – определяет «глубину» вершины и может использоваться при включенном тесте глубины. Данная переменная также должна лежать в диапазоне [-1…+1].



Для лучшего понимания, рассмотрим шейдер, использованный в демонстрационном примере:

```
1  #version 330 core
2
3  uniform vec2 offset;
4
5  in vec2 vPosition;
6
7  out vec2 position;
8
9  void main ()
10 {
11     position = vPosition;
12
13     gl_Position = vec4 (vPosition + offset, 0, 1);
14 }
15
```

Здесь необходимо обратить внимание на следующие основные моменты:

1. В первой строке происходит объявление используемой версии языка GLSL. В данном случае запрашивается версия GLSL 3.3. Если данная версия не поддерживается видеокартой, то она не сможет скомпилировать и использовать данный шейдер.
2. В третьей строке происходит объявление uniform-переменной, которая называется offset и которая является вектором из двух компонент. Значение данной переменной устанавливается из основной программы и здесь используется для того, чтобы сместить выводимый квадрат на определенное расстояние по горизонтали и вертикали. Значение данной переменной является одинаковым при каждом вызове вершинного шейдера, то есть общим для всех вершин.
3. В пятой строке происходит объявление входной переменной, которая является атрибутом вершины. Как было указанно выше, в данном примере каждая вершина имеет только один атрибут – геометрические координаты. При вызове данного вершинного шейдера эта переменная будет поочередно принимать координаты обрабатываемых вершин, например: (-0.5, +0.5), (-0.5, -0.5), (+0.5, +0.5) и так далее. Значение данной переменной является разным для всех вершин (кроме совпадающих, но это частный случай).
4. В седьмой строке происходит объявление выходной переменой. Данная переменная будет использоваться для передачи данных от вершинного шейдера к фрагментному.
5. В девятой строке происходит объявление функции main. Именно с этой функции начинается выполнение вершинного шейдера.
6. В одиннадцатой строке происходит вычисление данных, которые будут переданы во фрагментный шейдер. В данном случае в указанную переменную просто перезаписывается начальная позиция вершины (позиция без учета смещения). Зачем эти данные необходимы фрагментному шейдеру будет рассказано в следующем разделе.
7. В тринадцатой строке происходит заполнение встроенной переменной gl_Position, которая определяет положение вершины на экране. В данном случае положение вершины определяется её начальной позицией (уникальной для всех вершин) и некоторым смещением (общим для всех вершин). Для лучшего понимания рекомендуется представить, чему будет равно значение для каждой вершины, при различных значениях смещения. Например, если смещение равно (0.0, 0.0), (0.0, +0.5) или (+0.5, +0.5). Также необходимо отметить, что переменная gl_Position является вектором из четырех компонент и в данном случае компонента z всегда равна нулю, а компонента w всегда равна единице.

Работа фрагментного шейдера.

После обработки каждой вершиной OpenGL знает, в каком месте экрана должна располагаться данная вершина. После этого происходит процесс растеризации – то есть разбиение полигона на множество фрагментов (пикселей), каждый из которых будет обрабатываться фрагментным шейдером.

При этом в шейдер передается следующая информация:

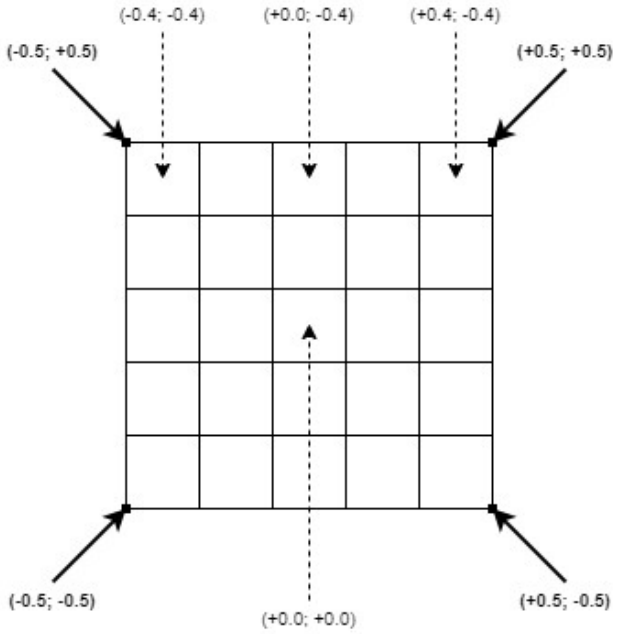
1. **Uniform-переменные**. Значения uniform-переменных были ранее установлены из основной программы и являются общими для всех фрагментов.
2. **Входные переменные** – это переменные, которые в вершинном шейдере были объявлены с квалификатором out, а во фрагментном шейдере с квалификатором in. Значения этих переменных будут отличаться для разных фрагментов, благодаря чему можно добиться вариативности их раскраски. Способ вычисления значений рассмотрен ниже.
3. **Дополнительные встроенные переменные.** Данные переменные генерируются самой видеокартой и могут использоваться в процессе обработки фрагментов.

Используя полученные данные, фрагментный шейдер должен выполнить две основные задачи:

1. **Определить выходные данные.** В простейшем случае указывается одна переменная, являющаяся вектором из четырех компонент (r, g, b, a), которая будет считаться цветом фрагмента. Значение, записанное в указанную переменную, будет использоваться OpenGL для модификации буфера кадра.
2. **Определить глубину фрагмента.** Фрагментный шейдер может заполнить встроенную переменную gl_FragDepth, явным образом указав глубину фрагмента. Указанная глубина будет участвовать в тесте глубины, если он активирован. В случае, если во фрагментном шейдере обращение к переменной gl_FragDepth не происходит, глубина фрагмента будет рассчитана автоматически, на основании глубин вершин, образующих полигон, которому принадлежит обрабатываемый фрагмент.

Особое внимание следует уделить способу передачи данных от вершинного шейдера во фрагментный. Для передачи таких данных вершинный шейдер объявляет out-переменные, а фрагментный in-переменные с одним и тем же названием и типом данных. При это прямая передача значений невозможна, поскольку количество вершин полигона и количество сгенерированных после растеризации фрагментов в общем случае отличается.

Для расчета значения входной переменной фрагментного шейдера используется линейная интерполяция, то есть значение переменной плавно меняется вдоль всего полигона от одной вершины к другой. Для лучшего понимания на рисунке снизу указаны значения, записанные в вершинном шейдере, и значения, сгенерированные для некоторых фрагментов:



Рассмотрим шейдер, использованный в демонстрационном пример:

```
1  #version 330 core
2
3  uniform vec4 color1;
4  uniform vec4 color2;
5
6  in vec2 position;
7
8  out vec4 fragColor;
9
10 void main ()
11 {
12     float gradient = position.x + 0.5;
13
14     fragColor = color1 * gradient + color2 * (1.0 - gradient);
15 }
```

Здесь необходимо обратить внимание на следующие моменты:

1. В первой строке происходит объявление используемой версии языка GLSL. В данном случае запрашивается версия GLSL 3.3. Если данная версия не поддерживается видеокартой, то она не сможет скомпилировать и использовать этот шейдер.
2. В третьей и четвертой строках происходит объявление uniform-переменных color1 и color2. Значения переменных устанавливаются из основной программы и здесь используются для того, чтобы определить цвет каждого фрагмента как комбинацию этих двух цветов, добившись, таким образом, градиентной закраски. Значения этих переменных являются одинаковым при каждом вызове фрагментного шейдера, то есть общими для всех фрагментов.
3. В шестой строке происходит объявление входной переменной position. Данная переменная была получена от вершинного шейдера, но не напрямую, а после растеризации и линейной интерполяции значений в вершинах полигона в соответствии с рисунком, представленным выше. Значение переменной является индивидуальным для каждого фрагмента и плавно меняется вдоль полигона.
4. В восьмой строке происходит объявление выходной переменой. Любая выходная переменная фрагментного шейдера трактуется как цвет, который должен быть записан в буфер кадра. В данном случае переменная является вектором из четырех компонент и названа fragColor. Теоретически фрагментный шейдер может вычислять несколько выходных переменных, но в этой работе данная возможность не используется.
5. В десятой строке происходит объявление функции main. Именно с этой функции начинается выполнение фрагментного шейдера.
6. В двенадцатой строке происходит объявление и инициализация локальной переменной gradient, типа float. Указанная переменная будет определять степень смешивания двух цветов и, как можно проследить по приведенному выше изображению, будет меняться от нуля до единицы для различных фрагментов.
7. В четырнадцатой строке происходит вычисление выходной переменной fragColor, которая задает цвет фрагмента. Значение данной переменной вычисляется как взвешенная сумму двух цветов. Например, если ранее вычисленное значение переменной gradient равно единице, то цвет фрагмента будет равен первому цвету. Если значение gradient равно нулю, то цвет фрагмента будет равен второму цвету. Для промежуточных значений переменной gradient цвет фрагмента будет плавно меняться от первого цвета ко второму, по мере того как для фрагментов будет меняться входная переменная position (x-компонента вектора).
8. Следует отметить, что в шейдере глубина фрагмента не определяется и поэтому вычисляется автоматически на основании глубины всех вершин (здесь также происходит линейная интерполяция глубин, заданной в каждой вершине).

Взаимодействие основной программы и шейдера.

При написании приложений трехмерной графики с использованием шейдеров следует внимательно подойти к проектированию архитектуры приложения. Также как и в проекте предыдущего семестра, программу можно разбить на несколько основных частей (хотя выделять отдельные модули пока не обязательно). В частности, сами шейдеры определяют алгоритм преобразования вершины и вычисления цвета фрагментов и для своей работы требуют передачи определенных данных из основной программы. Поэтому, прежде всего, необходимо определить глобальные переменные, влияющие на вывод изображения на экран. В демонстрационном примере для этого используются следующие переменные:

```
// ИСПОЛЬЗУЕМЫЙ ШЕЙДЕР (ПОКА ТОЛЬКО ОДИН)
Shader shader;

// ДАННЫЕ ДЛЯ ВЫВОДА ПРЯМОУГОЛЬНИКА
// текущее смещение прямоугольника
vec2 offset = vec2(0.25, 0);
// скорость (направление) перемещения прямоугольника
vec2 speed = vec2(+0.30, -0.25);
// первый цвет (для градиентной заливки)
vec4 color1 = vec4(1, 0, 0, 1);
// второй цвет (для градиентной заливки)
vec4 color2 = vec4(0, 0, 1, 1);
```

Далее необходимо реализовать функцию для вывода изображения на экран, которая будет вызываться всякий раз, когда требуется перерисовать окно. Данная функция должна использовать глобальные данные для построения изображения, но не должна их менять, поскольку это не входит в круг её обязанностей. Для вывода объекта (квадрата) используется функция drawObject из предыдущей лабораторной работы. Следует также обратить внимание, что в задании требуется организовать подсчет FPS (функция для подсчета FPS не приводится). Ниже приведен полный текст функции display:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
	// очистка буфера кадра
	glClearColor (1.0, 1.0, 1.0, 1.0);
	glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

	// включение теста глубины (на всякий случай)
	glEnable(GL_DEPTH_TEST);

	// активация шейдера и установка значения uniform-переменных
	shader.activate();
	shader.setUniform("offset", offset);
	shader.setUniform("color1", color1);
	shader.setUniform("color2", color2);

	// вывод квадрата
	drawObject();

	// смена переднего и заднего буферов
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_02 [%i FPS]", FPS);
	glutSetWindowTitle(windowTitle);
}
```

Затем необходимо определить функцию симуляции, которая должна вызываться максимально часто и изменять глобальные данные в соответствии с алгоритмом симуляции. Как правило симуляция должна происходить с заранее определенной скоростью, поэтому функция simulation должна определять время симуляции – время прошедшее с предыдущего шага симуляции. Пример реализации функции представлен ниже, текст функции для определения времени симуляции и сам алгоритм изменения глобальных данных не приведены:

```
// функция симуляции - вызывается максимально часто
// через заранее неопределенные промежутки времени
void simulation()
{
	// полученичe времени симуляции
	float simulationTime = getSimulationTime();

	// вычисление нового смещения и изменение прочих параметров
	offset = offset + ...
	...

	// установка признака необходимости перерисовать окно
	glutPostRedisplay();
};
```

И, наконец, функция main должна инициализировать все библиотеки, устанавливать функции обратного вызова и инициализировать начальные значение нужных переменных. Поскольку данные действия выполняются также, как это происходило в предыдущей работе, здесь функция main не приводится.

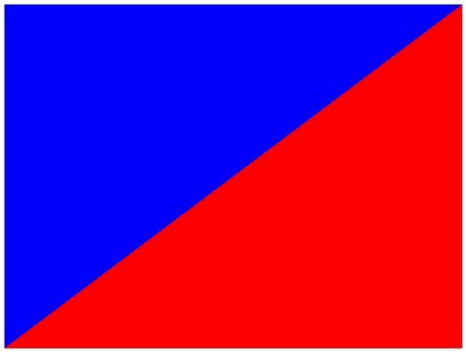
Способ реализации функций для расчета FPS и времени симуляции были описаны в лабораторных работах прошлого семестра и может быть позаимствован оттуда. Для студентов, которые желают получить дополнительный опыт программирования рекомендуется воспользоваться модулем ***std::chrono*** из стандартной библиотеки C++.

Задание к лабораторной работе.

Лабораторная работе №2 строится на основании предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо модифицировать класс Shader для работы с шейдером так, чтобы он содержал методы для эффективной установки uniform-переменных. Класс должен иметь структуру, приведенную в одном из предыдущих разделов.

2. Необходимо модифицировать шейдер так, чтобы прямоугольник был раскрашен одним из двух способов, с учетом группы студента, выполняющего работу:

Для группы ИВТ 41-18	Для группы ИВТ 42-18
	

3. Цвета для раскраски объекта передаются через uniform-переменные и задаются в основной программе в виде констант, которые, при необходимости, можно изменить.

4. Необходимо реализовать автоматическое перемещение объекта так, как это сделано в демонстрационном примере: в начальный момент времени объект движется в произвольном направлении, достигнув границ экрана объект «отскакивает» от них и начинает двигаться в другом направлении. Начальное положение объекта и скорость задаются в основной программе в виде переменных, значения которых, при необходимости, можно изменить.

5. Необходимо сделать так, чтобы перемещение быть плавным и не зависело от производительности компьютера, то есть в функции симуляции необходимо определить время симуляции и пропорционально изменить значение переменной offset. Время симуляции определяется так же, как и в проекте предыдущего семестра.

6. Кроме того, в лабораторной работе необходимо рассчитать FPS и вывести его в заголовке окна. Расчет FPS выполняется также, как и в лабораторных работах прошлого семестра.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №2 необходимо организовать вывод перемещающегося прямоугольника, раскрашенного указанным способом. В процессе выполнения работы должен быть написан класс Shader.

Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Что такое шейдер и в чем отличие шейдера от обычных программ.
2. **[КГ]** В чем заключается работа вершинного шейдера.
3. **[КГ]** В чем заключается работа фрагментного шейдера.
4. **[КГ]** Что такое uniform-переменные и как происходит установка значений uniform-переменных.
5. **[КГ]** Как происходит передача данных между вершинным и фрагментным шейдерами.
6. **[CPP]** Что такое контейнер std::map и как выполняются основные операции с этим контейнером (создание, инициализация, поиск элемента, добавление элемента, получение значения по ключу и удаление элемента).
7. **[CPP]** Что такое статические поля класса, как происходит объявление и инициализация этих полей, а также в чем отличие статических полей класса от обычных.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		